

b. Success or failure tree? Finite or infinite?

1. **Success tree:** the tree contains success paths, in fact, all paths are in branch A.
2. **Infinite tree:** there are two independent sources of infiniteness:
 - **Branch A:** unary_number(X) with X free enumerates every unary number (left branch).
 - **Branch B:** unary_plus([1|X], T, X) recurses forever, while fails on the occurs-check at every level.

Hence, by the course material this is an infinite success tree, an infinite tree that contains a finite success path.

c. Is the query provable?

Yes., since there exists a finite success path from the root to a success leaf for example the path ending in $\{X = [], Y = []\}$.

d. Is L9 decidable?

No, L9 is not decidable. The relational language (no functors) is decidable because there's only a finite number of possible queries, so if we ever reach a query we've already seen, we know it's a loop that will never hit a success node, and we can just answer "not provable". Once we add even one functor like cons, this stops working: now we can build bigger and bigger terms (cons(a,a), cons(a,cons(a,a)), ...), so a program can keep generating new goals that never repeat. For example, $p(X) :- p(\text{cons}(a,X))$ with the query $?- p(a)$ keeps going forever, producing a larger goal every step and never returning to an old node, so the "we came back to the same node" trick simply never triggers. The "rational" part doesn't save it either: rational trees only make unification work on cyclic terms like $X = \text{cons}(a,X)$ (instead of failing on the occurs-check), but they don't stop the regular, acyclic terms from growing without bound.